

Module 7: Single-Dimensional Arrays

Sarah Foss

Objectives

Learn how to declare, create, and initialize arrays in Java using the appropriate syntax, and recognize the difference between declaring an array and allocating memory for it.

Understand how array indexing works, starting from 0, and identify the default values assigned to array elements based on their data type.

Learn how arrays are passed to methods by reference and how this allows methods to modify the original array's elements.

Understand and implement basic sorting (selection sort) and searching (binary search) techniques, and recognize when each is most efficient.

Arrays Overview

Suppose you need many variables in your program.

You could either create a separate name for each variable:

```
double d1, d2, d3, d4, d5;
```

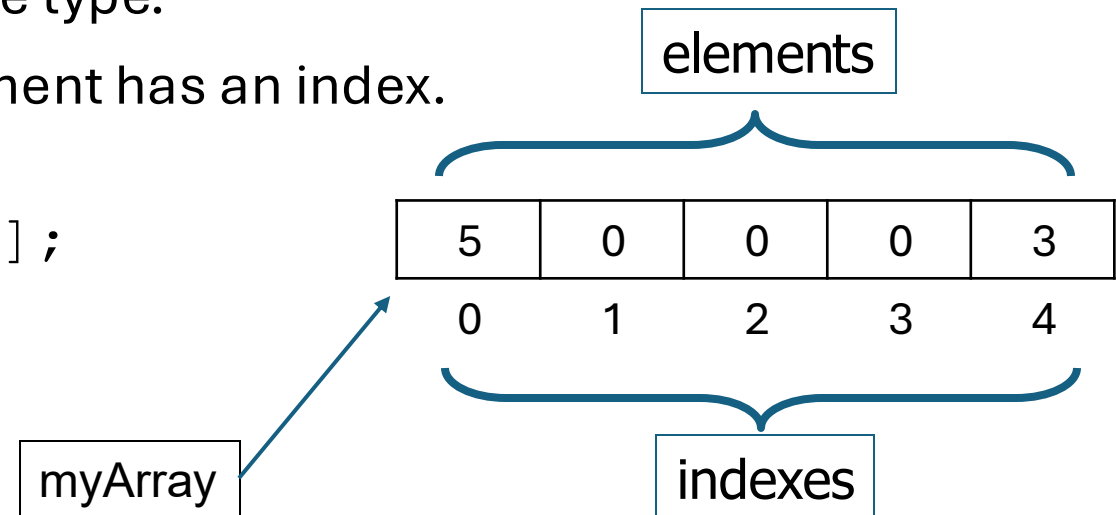


Or you could create an **array**.

An array is a collection of data items of the same type.

An array has multiple **elements**, and each element has an index.

```
double[] myArray = new double[5];  
myArray[0] = 5;  
myArray[4] = 3;
```



Declaring Arrays

An array is a collection of elements, all of the same type, stored in a contiguous block of memory.

Declaring syntax:

```
datatype[] arrayName;
```

Example:

```
int[] numbers;
```

The variable will hold a reference to an array object in memory. However, until you actually create the space for the array using `new` (or curly braces), **no array exists in memory.**

Initializing Arrays

There are two main ways to initialize an array in Java:

Using the **new** keyword:

```
numbers = new int[10];
```

Initializing with values:

```
numbers = {9, 8, 7, 6, 5, 4, 3, 2, 1, 0};
```

Until you actually create the space for the array using `new` (or curly braces), **no array exists in memory**.

The size of an array is fixed once it's created.

Array Indexing

When creating an array using `new`, the number in square brackets is the number of elements in the array:

```
int[] arr = new int[20]; // 20 elements
```

The first spot of the array has index 0 instead of 1.

To access or set a value in an array, use its index:

```
arr[0] = 10;           // Sets first spot to 10
arr[19] = arr[0];      // Sets last element same as 1st
int value = arr[2];    // Retrieves the value from the element at index 2
```

Default Values

When an array is created using the `new` keyword, its elements are assigned the default value of :

- 0 for the numeric primitive data types,
- '\u0000' for char types, and
- false for boolean types.

Array Length

The `length` property gives the size (number of elements) of the array.

```
int length = numbers.length;
```

Since the index of an array always begins with 0, the last index is always:
`arrayName.length - 1`.

An out-of-bounds error will occur if you attempt to reference elements beyond the bounds of an array.

Looping through Arrays

You can use loops to iterate over array elements:

```
for (int i = 0; i < numbers.length; i++) {  
    System.out.println(numbers[i]);  
}
```

Trace Through an Array

```
public class Test {  
    public static void main(String[] args) {  
        int[] values = new int[5];  
        for (int i = 1; i < 5; i++) {  
            values[i] = i + values[i-1];  
        }  
        values[0] = values[1] + values[4];  
    }  
}
```

Common Operations:

Assigning Input Values to Arrays

```
Scanner input = new Scanner(System.in);  
int[] myArray = new int[5];  
for (int i = 0; i < myArray.length; i++) {  
    System.out.print("Enter a number: ");  
    myArray[i] = input.nextInt();  
}
```

Common Operations:

Displaying Array Contents

```
for (int i = 0; i < myArray.length; i++) {  
    System.out.print(myArray[i] + " ");  
}  
System.out.println();
```

Common Operations:

Populating Arrays with Random Values

```
int[] randomNumbers = new int[10];  
for (int i = 0; i < randomNumbers.length; i++) {  
    randomNumbers[i] = (int) (Math.random() * 100);  
}
```

Common Operations:

Calculating the Sum of Array Elements

```
int sum = 0;
for (int i = 0; i < myArray.length; i++) {
    sum += myArray[i]
}
```

Common Operations:

Finding largest element

```
int largest = myArray[0];  
for (int i = 1; i < myArray.length; i++) {  
    if(myArray[i] > largest){  
        largest = myArray[i];  
    }  
}
```

Common Operations:

Locating the Smallest Index of the Largest Element

```
int largest = myArray[0];
int index = 0;
for (int i = 1; i < myArray.length; i++) {
    if(myArray[i] > largest){
        largest = myArray[i];
        index = i;
    }
}
```


Common Operations:

Randomly Shuffling Array Elements

```
for (int i = 0; i < myArray.length; i++) {  
    int random = (int) (Math.random() * myArray.length);  
    int temp = myArray[i];  
    myArray[i] = myArray[random];  
    myArray[random] = temp;  
}
```

Common Operations: Shifting Array Elements

```
int first = myArray[0];  
for (int i = 0; i < myArray.length-1; i++) {  
    myArray[i] = myArray[i+1];  
}  
myArray[myArray.length-1] = first;
```

For Each Loop

The **for-each loop** (also known as the **enhanced for loop**) is a simple way to iterate over elements in arrays or collections without needing to manage index counters.

It's primarily used when you don't need to modify the elements or know their index, and it automatically handles the traversal of the array.

Syntax:

```
for (dataType element : array) {  
    // Code to process each element  
}
```

Examples:

```
for (int x : myArray) {  
    System.out.print(x + " ");  
}
```

For Each Loop Notes

- Reads every element in the array from the beginning to the end.
- Does not give access to the index of the element. If you need the index, you'll have to use a traditional for loop.
- Cannot be used to modify elements of an array directly (modifying the element variable does not affect the array).
- Works well for cases where you only need to read the array, not change it.

Copying Arrays

In many programs, you may need to copy an entire array or a portion of it.

You might try using the assignment operator (=) for this, but that may have unexpected results:

```
int[] original = {1, 2, 3, 4, 5};  
int[] copy = original;
```

This is because both variables are now holding a **reference** to the same array in memory. No new array has been created.

Copying Arrays

To actually create a copy of an array, can manually copy the elements from one array to another using a for loop:

```
int[] original = {1, 2, 3, 4, 5};  
int[] copy = new int[original.length];  
for (int i = 0; i < original.length; i++) {  
    copy[i] = original[i];  
}
```

Passing Arrays to Methods

- When you pass an array to a method in Java, you are actually passing a copy of the reference to the array, not the array itself.
- This means that any changes made to the array inside the method will affect the original array because both the original array and the array inside the method point to the same memory location.

Passing Arrays to Methods

```
public class ArrayTest {  
    public static void main(String[] args) {  
        int[] arr = {10, 20, 30};  
        modifyArray(arr);    System.out.println(arr[0]);  
    }  
    public static void modifyArray(int[] array) {  
        array[0] = 100; // Modifies the original array  
    }  
}
```


Pass-By-Value

Java is pass-by-value: Whether it's a primitive or a reference, a copy of the value is passed to the method.

- When primitive data types (like `int`, `char`, `double`) are passed to a method, a copy of the actual value is passed. *Changing this value in the method does not affect the original variable.*
- When objects (like arrays) are passed to a method, a **copy of the reference** to the object is passed. This means the reference to the array is passed by value, but since the reference points to the same memory location, *the method can modify the elements of the original array.*

Heap

- The **heap** is a runtime memory area where the JVM dynamically allocates memory for objects and arrays.
- Objects (including arrays) live in the heap until they are no longer referenced, at which point Java's **garbage collector** reclaims the memory.
- Local variables in Java (like array variables) are stored in the **stack** (a separate memory area), but they contain references (or pointers) to the actual arrays, which are stored in the heap.

Returning an Array from a Method (Trace)

```
public static int[] reverse(int[] list) {  
    int[] result = new int[list.length];  
  
    for (int i = 0, j = result.length - 1;  
        i < list.length; i++, j--) {  
        result[j] = list[i];  
    }  
  
    return result;  
}
```

Anonymous Array

An anonymous array is an array that is created without assigning it to a specific variable.

These arrays are used in situations where you need to pass an array directly to a method or use it immediately without needing to store or reference it elsewhere.

Since they are unnamed, they exist temporarily and are usually used in a single place in the code.

Syntax:

```
new dataType[] { element1, element2, ..., elementN }
```

Example:

```
printArray(new int[] {1, 2, 3, 4, 5});
```

Variable-Length Arguments

A variable number of arguments of the same type can be passed to a method and treated as an array.

The parameter in the method is declared as follows:

```
typeName . . . parameterName
```

In the method declaration, you specify the type followed by an ellipsis (...).

Only one variable-length parameter may be specified in a method, and this parameter must be the last parameter. Any regular parameters must precede it

Searching Arrays

Searching is the process of finding a specific element (or elements) within an array or collection.

There are different ways to search for elements in an array, depending on the nature of the data and whether the array is sorted or unsorted.

Searching is a common task in computer programming.

There are two common searching techniques:

- Linear Search
- Binary Search

Linear Search

In linear search, you start at the beginning of the array and check each element one by one until you find the desired element or reach the end of the array.

```
public static int linearSearch(int[] arr, int key) {  
    for (int i = 0; i < arr.length; i++) {  
        if (arr[i] == key) {  
            return i;  
        }  
    }  
    return -1;  
}
```

Binary Search

Binary search is a much faster search algorithm, but it only works for **sorted** arrays.

It works by repeatedly dividing the search space in half:

- First, it checks the middle element of the array.
- If the middle element is equal to the target, the search ends.
- If the target is smaller than the middle element, it searches in the left half of the array.
- If the target is larger, it searches in the right half.
- This process is repeated until the element is found or the search space is reduced to zero.

Binary Search

```
public static int binarySearch(int[] arr, int key) {  
    int low = 0;  
    int high = arr.length - 1;  
    while (low <= high) {  
        int mid = (low + high) / 2;  
        if (arr[mid] == key) {  
            return mid; // Element found  
        } else if (arr[mid] < key) {  
            low = mid + 1;  
        } else {  
            high = mid - 1;  
        }  
    }  
    return -1; // Element not found  
}
```

Built in Searching Methods

The `java.util.Arrays` class provides a built-in `binarySearch` method that you can use to perform binary searches more easily.

```
int[] arr = {2, 3, 4, 7, 8, 10};  
int target = 7;  
int index = java.util.Arrays.binarySearch(arr, target);
```

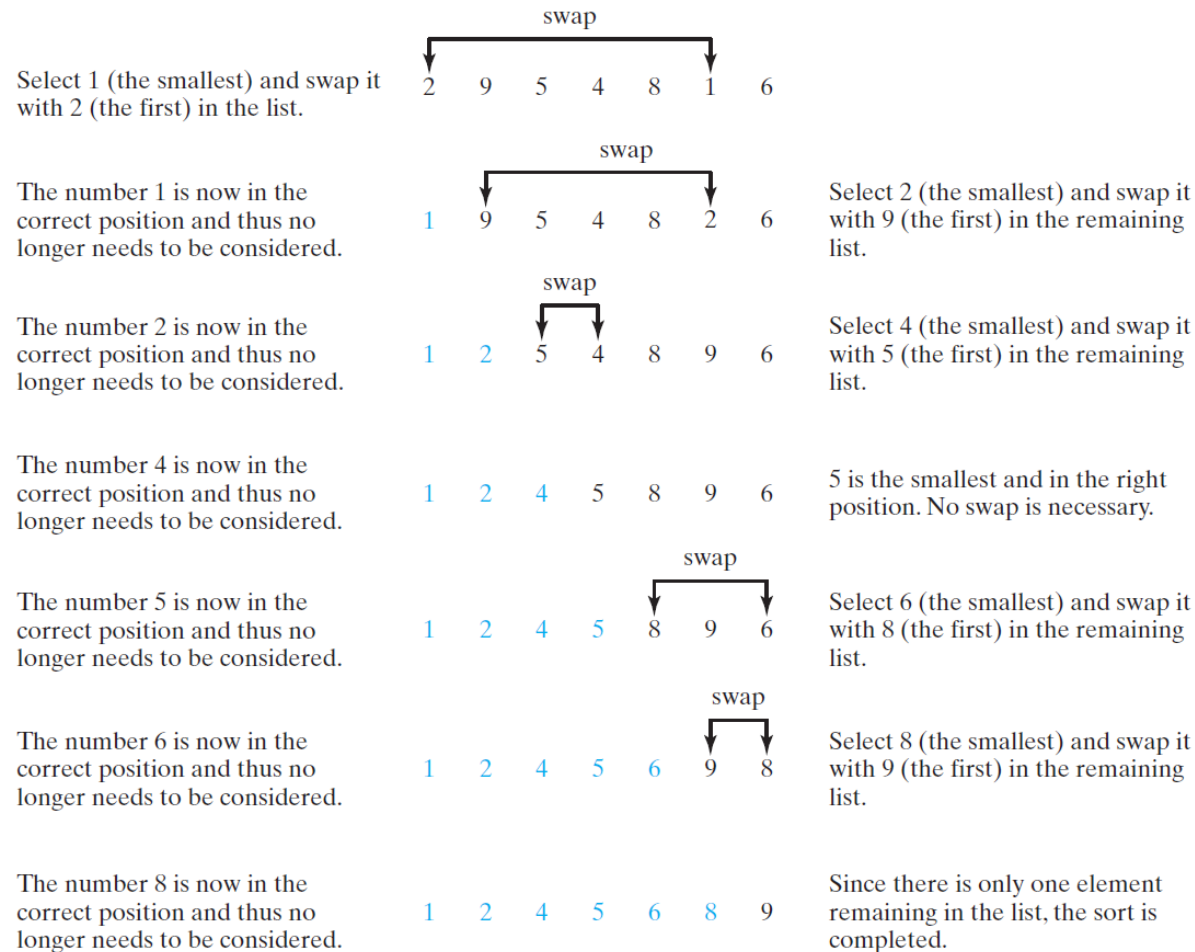
Sorting Arrays

Sorting is the process of arranging elements in an array or collection in a particular order, usually either in ascending or descending order.

A simple algorithm for sorting is **Selection sort**.

Selection Sort

Selection sort finds the smallest number in the list and places it first. It then finds the smallest number remaining and places it second, and so on until there is only one unsorted number left.



Selection Sort

```
public static void selectionSort(int[] arr) {  
    int n = arr.length;  
    for (int i = 0; i < n - 1; i++) {  
        int minIdx = i;  
        for (int j = i + 1; j < n; j++) {  
            if (arr[j] < arr[minIdx]) {  
                minIdx = j; // Find the minimum element  
            }  
        }  
        // Swap the found minimum with the first element  
        int temp = arr[minIdx];  
        arr[minIdx] = arr[i];  
        arr[i] = temp;  
    }  
}
```

Built-in Sorting Methods

Java provides built-in sorting methods in the `java.util.Arrays` class.

These methods use efficient sorting algorithms and are highly optimized for performance.

```
int[] arr = {3, 5, 1, 4, 2};  
java.util.Arrays.sort(arr);
```

Command Line Arguments

- Command-line arguments are inputs passed to a program at the time of execution via the command line or terminal.
- These arguments can provide runtime configuration or data to the program without hardcoding values in the source code.
- Programs access command-line arguments through the `args[]` array in the main method (If no arguments are passed, the array will be empty).

Command Line Arguments

```
public class CommandLineExample{  
    public static void main(String[] args) {  
        for(String s: args) {  
            System.out.println(s);  
        }  
    }  
}
```

To run (on the command line):

```
javac CommandLineExample.java
```

```
java CommandLineExample Java is awesome
```


Summary

- An array in Java is declared using `type[] var` (preferred) or `type var[]`.
- Declaring an array variable does not allocate memory; it just holds a reference to an array.
- Arrays must be created with the `new` operator before assigning values.
- Array indices start at 0, and accessing elements outside the bounds causes an error.
- Arrays have default values: 0 for numbers, `\u0000` for char, and null for object types.
- Java allows array initializers to declare, create, and initialize arrays in one step.
- When passing an array to a method, you pass a reference, so the original array can be modified.
- Binary search is more efficient than linear search for sorted arrays.
- Selection sort works by finding the smallest element and swapping it with the first unsorted element repeatedly.